

Week 2 — Supervised ML: Regression

Predicting the daily high temperature from weather data.

What machine learning is

- A model learns patterns from data.
- We do not write the rules by hand.
- The learned rules are called the model.
- More data usually means better patterns.

Supervised learning

- The training data includes known answers.
- The model learns inputs to output.
- Two kinds: regression and classification.
- This week we cover regression.

Regression

- Regression predicts a continuous number.
- Examples: a price, a duration, a temperature.
- Our target is the daily high temperature.
- Classification (a category) comes in Week 3.

The scikit-learn API

- Every model follows the same three steps.
- Create the model.
- Fit it to the training data.
- Predict on new inputs.

create → fit → predict

```
model = LinearRegression()    # create  
model.fit(X_train, y_train)   # learn  
y_pred = model.predict(X_test) # predict
```

Features must be 2D

- `X` must have shape `(n_samples, n_features)`.
- Each row is one sample.
- Each column is one feature.
- One feature still needs a column.

Reshape a single feature

```
x = np.array([10, 20, 30, 40, 50])  
print(x.shape)           # (5,)  
X = x.reshape(-1, 1)  
print(X.shape)           # (5, 1)
```


Why split the data

- We want to know how the model does on new data.
- Training data alone can be memorized.
- We hold part of the data aside to test.
- This is how we measure generalization.

Train/test split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

- 80 percent trains, 20 percent tests.
- `random_state` makes the split repeatable.

Linear regression on weather

- Predict the high from the low temperature.
- The model learns a slope and an intercept.
- $\text{temp_max} = \text{slope} * \text{temp_min} + \text{intercept}$.
- Cold nights tend to go with cold days.

Reading the slope

- The slope is the most useful number.
- It is the rise in the high per degree of the low.
- On this data the slope is close to 1.0.
- The high rises about one degree per degree of low.

Evaluating with RMSE

- RMSE is the typical size of the error.
- It is reported in the original units.
- Here that is degrees Celsius.
- A low temperature model gives RMSE near 2.3.

Evaluating with R^2

- R^2 compares the model to guessing the mean.
- 1.0 is perfect. 0.0 is no better than the mean.
- R^2 can go below 0 on test data.
- A one-feature weather model reaches about 0.92.

Adding more features

- Pass more columns as `X`.
- The same fit and evaluation code still works.
- One feature fits a line. Two fit a plane.
- More features fit a hyperplane.

Interpreting coefficients

- Each coefficient is one feature's effect.
- It holds the other features constant.
- Rain and wind coefficients come out negative.
- `is_summer` adds about 3 degrees when it is 1.

A weak feature can still help

- `is_summer` correlates weakly with the high on its own.
- Its correlation is only about 0.19.
- In the full model its coefficient is clear.
- The low temperature hid the seasonal signal.

Overfitting and underfitting

- Underfitting: the model misses the real trend.
- Overfitting: the model memorizes the noise.
- Watch the gap between train R^2 and test R^2 .
- A large gap means the model overfit.

Why linear models resist overfitting

- A line or plane has limited flexibility.
- It cannot bend to every training point.
- This makes it a safe first model.
- More flexible models need more caution.

Looking ahead to Week 3

- We keep the same daily weather data.
- The task changes to classification.
- We predict a category instead of a number.
- The `create → fit → predict` API stays the same.